

Plato: Generic Ontology-Based Representation of Semantic Planning Knowledge for Automatic PDDL Generation

Keywords: AI Planning, PDDL, Ontology Engineering, Semantic Planning Knowledge, Knowledge Engineering, Planning Domain Generation

Abstract: Artificial Intelligence (AI) planning can support automated decision-making in domains such as manufacturing, robotics, logistics, and autonomous systems. However, practical use requires explicit planning models, which are often created and maintained directly in the Planning Domain Definition Language (PDDL). This makes planning knowledge difficult to model, maintain, reuse, evolve, and adapt to different planners. This paper presents Plato, a generic ontology schema for representing semantic planning knowledge in a way that makes it easier to model, maintain, and adapt to different planners. Plato models central planning concepts such as typed entities, predicates, and actions as semantic objects. It is designed as a maintainable source representation from which planner-specific PDDL artifacts can be generated automatically. The ontology is supported by the Knowledge Transformation Framework, which transforms Plato ontologies into PDDL domain and problem files or even other semantic data structures. The proof-of-concept evaluation uses four example ontologies covering classical planning, temporal planning, and numeric planning with automatic discretization. The generated PDDL artifacts and plans are validated with VAL, and the planning tasks are solved with off-the-shelf planners. The results provide initial evidence that Plato can serve as a semantic source model for automatic planning-domain generation.

1 INTRODUCTION

Digital transformation is moving decision processes toward data- and knowledge-driven automation. Intelligent systems must increasingly generate, revise, and coordinate decisions under, for example, temporal, resource, or cost constraints, often while reacting to disruptions. Such requirements occur in various domains including autonomous space operations and planetary exploration (Muscettola et al., 1998; Bresina et al., 2005), urban planning (McCluskey et al., 2017), industrial manufacturing (Malburg et al., 2023; Wally et al., 2019), robotics (Alterovitz et al., 2016), and video games (Barthelemy and Jacopin, 2009).

Within this broader context, Artificial Intelligence (AI) methods such as AI planning (Green, 1969; Haslum, 2006) addresses such problems. AI planning automatically constructs action sequences or schedules that achieve goals while respecting constraints. A planning application typically combines a model, a representation language, and a planner (Marrella, 2019). The model describes the domain; the language encodes this information; and the planner computes a solution. In practice, Planning Domain Definition Language (PDDL) has become the de facto standard representation language (Haslum et al., 2019;

Helmert, 2009; Ammar and Bhiri, 2018) for this purpose.

Although AI planning can support autonomous decisions (Muscettola et al., 1998; Bresina et al., 2005), increase process flexibility (Malburg et al., 2023), and enable replanning under failures or changing conditions (Kootbally et al., 2015a), its practical use remains limited. A central obstacle is the acquisition, formalization, and maintenance of planning knowledge. Planning is knowledge-intensive (Alnazer and Georgievski, 2023), and effective deployment requires detailed domain knowledge encoded in explicit models (Malburg et al., 2023). Creating these models reflects the knowledge acquisition bottleneck of knowledge-based AI: formalizing expert knowledge is costly, time-consuming, and error-prone (Feigenbaum, 1977; Studer et al., 1998), especially as domains evolve (Studer et al., 1998).

Malburg et al. (Malburg et al., 2023) addressed this bottleneck by maintaining planning-relevant knowledge in an ontology and transforming it automatically into PDDL domain and problem descriptions. Their approach reduces manual PDDL modeling and supports planner-oriented transformations, such as substituting numeric expressions for planner compatibility. However, it was developed for a Fischertechnik model factory and does not provide a se-

semantic representation for generic planning domains or a broad range of planning constructs.

This paper addresses this limitation with Plato: The Planning Ontology, a generic ontology schema for representing semantic planning knowledge independently of specific application cases. Plato captures the information required for automatic PDDL generation while remaining reusable across heterogeneous planning environments. It can serve as a easier maintainable semantic source model from which planner-ready PDDL representations are generated. The paper contributes a requirement-driven ontology model for semantic planning knowledge and demonstrates its feasibility to serve as a semantic representation of planning knowledge for automated PDDL generation in a proof-of-concept evaluation in which example Plato ontologies are transformed into PDDL domains using the Knowledge Transformation Framework (KTF) (KTF, 2026).

The remainder is organized as follows. Section 2 introduces background and related work. Section 3 presents requirements and methodology. Section 4 describes Plato, and Section 5 covers automatic PDDL generation. Section 6 reports the evaluation. Section 7 discusses contributions and limitations, and Section 8 concludes the paper.

2 FOUNDATIONS AND RELATED WORK

This section summarizes the required planning background and positions Plato relative to related work. Basic ontology concepts such as Resource Description Framework (RDF), RDF Schema (RDFS), and Web Ontology Language (OWL) are omitted because of space constraints and the expected KEOD audience.

2.1 AI Planning and PDDL

AI planning determines how a system can reach a goal from an initial state (Green, 1969; Haslum, 2006). In classical planning, a state describes relevant world properties, and actions define state transitions through applicability conditions and effects. A planning problem consists of an initial state, a goal state, and available actions. A solution is usually a finite sequence of parameterized actions that transforms the initial state into the goal state.

The Blocks World domain (Russell and Norvig, 2020) illustrates these concepts. Blocks on a table can be picked up or stacked only under certain conditions, for example when no block lies on top of the block

to be moved. Executing an action adds and removes facts such as whether a block is held or stacked on another block. Although simple, the domain contains the same basic elements as larger applications including typed objects, predicates, actions, preconditions, and effects.

Planning Domain Definition Language is the de facto standard for representing planning domains and problems (Haslum et al., 2019; Helmert, 2009; Ammar and Bhiri, 2018). A PDDL model is usually split into a domain file, which defines reusable types, predicates, functions, constants, and actions, and one or more problem files, which instantiate objects, initial facts, and goals. Predicates encode Boolean state properties; functions encode numeric information; and actions specify parameters, preconditions, and effects.

Beyond classical planning, PDDL supports richer constructs. Temporal planning uses durative actions whose conditions and effects may hold at the start, end, or throughout the action (Fox and Long, 2003). Numeric planning uses functions and arithmetic expressions for costs, durations, and resources. Other constructs support advanced logical conditions, metrics, and constraints. Planner support for these fragments varies substantially, so a valid domain may be unusable with a particular planner.

This issue provides another motivating factor for Plato. Maintaining planning knowledge directly in PDDL couples domain knowledge to syntax and planner feature support. If a different planner or PDDL fragment is required, models often need manual adaptation. Plato instead represents planning knowledge in a semantic source model from which planner-specific PDDL artifacts can be generated and adapted during transformation.

2.2 Related Work

The difficulty of authoring and maintaining planning models has led to approaches that represent planning knowledge at a higher abstraction level and transform it into languages such as PDDL. These include model-driven engineering environments, semantic web service composition, and ontology-based planning models.

A prominent model-driven approach is itSIMPLE (Vaquero et al., 2013; Vaquero et al., 2007), which uses Unified Modeling Language (UML) and Object Constraint Language (OCL) to model planning information and maps it to PDDL (Vaquero et al., 2013). In the NIST APRS project, Kootbally et al. represent PDDL domain and problem structures in OWL and XML Schema Definition Language

(XSDL) for robotic kitting and assembly (Kootbally et al., 2015b). SysML-based approaches add stereotypes for PDDL concepts such as types, predicates, functions, and actions (Nabizada et al., 2024). These approaches integrate planning with engineering workflows but are often closely aligned with PDDL syntax or a specific domain, limiting the modeling relief offered by more abstract generic models.

A second line of work stems from semantic web service composition. OWL-S describes services through Inputs, Outputs, Preconditions, and Effects (IOPEs) (Martin et al., 2004), a structure close to planning because services can be interpreted as actions. OWLS-Xplan transforms OWL-S descriptions into PDDL 2.1 and uses a planner for service composition (Klusch et al., 2005). PORSCHE II adds semantic matching, relaxation, and reverse transformation (Hatzi et al., 2012), and Puttonen et al. apply related ideas to manufacturing (Puttonen et al., 2013). These works show the usefulness of semantic action descriptions, but they target service composition rather than generic planning-domain modeling.

Ontology-based approaches are closest to Plato. The Knowledge EngineeringWeb Interface (KEWI) represents domain knowledge in an ontology-centered environment and can export PDDL (Wickler et al., 2014). Ontology-mediated planning combines PDDL with OWL ontologies and rewrites the result into standard planning tasks (John and Koopmann, 2023). Such work connects ontological knowledge and planning, but often focuses on reasoning over enriched states, extending existing PDDL models, or organizing planner metadata rather than using an ontology as the primary source for domain generation.

The most direct predecessor is the approach by Malburg et al. (Malburg et al., 2023). Their SWS2PDDL converter uses semantic web services and a domain ontology to generate PDDL for manufacturing process planning and scheduling. It supports the most central PDDL constructs such as types, predicates, and actions, and includes transformations for different PDDL levels and planner capabilities. Their evaluation in a model factory showed that generated domains can produce plans comparable to or better than manually modeled domains. However, the source model is tied to a specific application case and has limited expressiveness for commonly used PDDL fragments. Plato generalizes this idea through a domain-independent ontology schema and adds constructs, such as discretization and modular costs, that can be translated into planner-specific PDDL expressions.

Positioning of Plato Prior work shows that semantic and model-driven representations can support planning-domain engineering and PDDL generation. However, many approaches are domain-specific, service-composition-specific, or closely coupled to explicit PDDL structures. Others enrich states, analyze existing artifacts, or organize metadata rather than serving as the primary source model for domain generation.

Plato targets a different design point: a generic ontology schema for planning-relevant knowledge itself. Types, entities, parameters, predicates, actions, preconditions, effects, functions, costs, metrics, and numeric constants are modeled as semantic objects. Plato also includes transformation-relevant constructs beyond direct PDDL serialization, such as information about how numeric values can be discretized and how modular action costs can be assembled during transformation. It is therefore not merely a PDDL serialization and is not limited to one application domain, but acts as a maintainable semantic source for generating planner-ready PDDL.

3 REQUIREMENTS AND METHODOLOGY

Plato was developed as a requirements-driven ontology schema for planning knowledge independent of a specific domain. Requirements were derived from three sources. First, we considered the most commonly used PDDL modeling constructs because Plato should serve as the semantic source for generated domain and problem files. Second, we built on ontology to PDDL transformation work, especially Malburg et al. (Malburg et al., 2023), which shows that a source model must contain action semantics and support transformations into different PDDL fragments. Third, we considered practical planner support and modeling experience. Since PDDL contains many rarely used or weakly supported constructs, Plato focuses on information commonly needed for domain modeling and automatic generation of planner-compatible artifacts. Community resources such as Planning.wiki¹ were used as practical guidance on constructs and planner support, not as formal standards.

The requirements define a pragmatic extensible core rather than a complete formalization of all PDDL variants. They cover symbolic domain structure, action semantics, numeric and optimization information, and transformation concerns caused by hetero-

¹<https://planning.wiki/>

geneous planner support.

R1: Typed planning entities and type hierarchies. Plato must represent objects, constants, parameters, and other entities in a domain or problem. Each entity needs a name and a type. Type hierarchies express generalization and specialization, reduce repeated modeling, and support typed PDDL generation.

R2: Predicates with ordered parameters. Plato must represent predicates that can describe world-state facts, including properties of entities and relations between them. Predicates need names and ordered typed parameters to enable unambiguous PDDL predicate definitions.

R3: Actions with parameters, preconditions, effects, and optional temporal information. Plato must represent action-like capabilities that transform the world state. Actions require typed parameters, applicability preconditions, and state-changing effects, which can include costs. This captures the planning-relevant part of the IOPEs principle. For temporal planning, the model should allow conditions and effects scoped to the start, end, or full duration of an action.

R4: Arithmetic expressions for costs and durations. Planning domains often require numeric action information such as costs or durations. Such expressions may be fixed values or composed arithmetic expressions involving constants, parameters, and function calls. Costs may also depend on non-numeric parameters such as machines, materials, locations, or transport resources. These kind of constructs must be supported by Plato.

R5: Optimization metrics. Plato must support optimization objectives that guide plan selection, from minimizing or maximizing one cost function to weighting several functions in an arithmetic expression. Many applications require not merely a valid plan but one preferred with respect to time, energy, resource use, monetary cost, or similar criteria.

R6: Functions and initial numeric values. Plato must represent numeric functions and initial values, for example parameter-dependent costs, durations, capacities, resource levels, or other numeric properties. It must capture both function signatures with names and parameters and concrete values for relevant assignments so that numeric information is maintained explicitly rather than embedded in actions or transformation code.

R7: Numeric discretization for planner compatibility. Bounded numeric information should be representable by Plato so that it can be transformed into discrete alternatives. This is necessary because many planners support restricted numeric frag-

ments or no general numeric reasoning, and continuous ranges can produce very large or infinite search spaces. Discretization preserves ranges and step sizes while enabling finite planner-compatible domains.

R8: Transformation into planner-specific PDDL fragments. Plato must support information for the generation of PDDL artifacts adapted to a target planner. Since planners differ in fragment support, the source model should not be tied to one representation. The transformation should emit different variants or simplify unsupported constructs, for example by replacing numeric expressions or rewriting logical constructs. Therefore, Plato should allow transformation-relevant semantic information that goes beyond one-to-one PDDL encoding, such as references to numeric value ranges from which discrete statements can be generated.

The requirements guided the ontology design in Section 4 and the proof-of-concept evaluation in Section 6. The methodology follows an artifact-oriented design process. We first identified planning information that must be represented semantically, then designed Plato as an ontology schema for that information, and finally evaluated it through transformation-oriented examples. In the evaluation, Plato ontologies are transformed into PDDL with KTF; the generated artifacts are inspected against the requirements and validated with planning tools. The evaluation is not intended to prove completeness for the full PDDL family, but to show whether Plato represents the selected core requirements and operationalizes them through automatic generation of planner-ready PDDL artifacts.

4 PLATO

This section introduces the core concepts of Plato. Plato is open source, and all resources and tooling are available under the MIT license on GitHub². Because of space constraints, only the main concepts are described here; a detailed specification is available in the repository.

4.1 Overview

Plato is an RDF/RDFS- and OWL-based ontology schema. It provides classes, object properties, data properties, annotation properties, and individuals for representing semantic planning knowledge. Its purpose is not to replace planning languages such as PDDL, but to provide a maintainable source model

²<https://github.com/user140613/Plato>

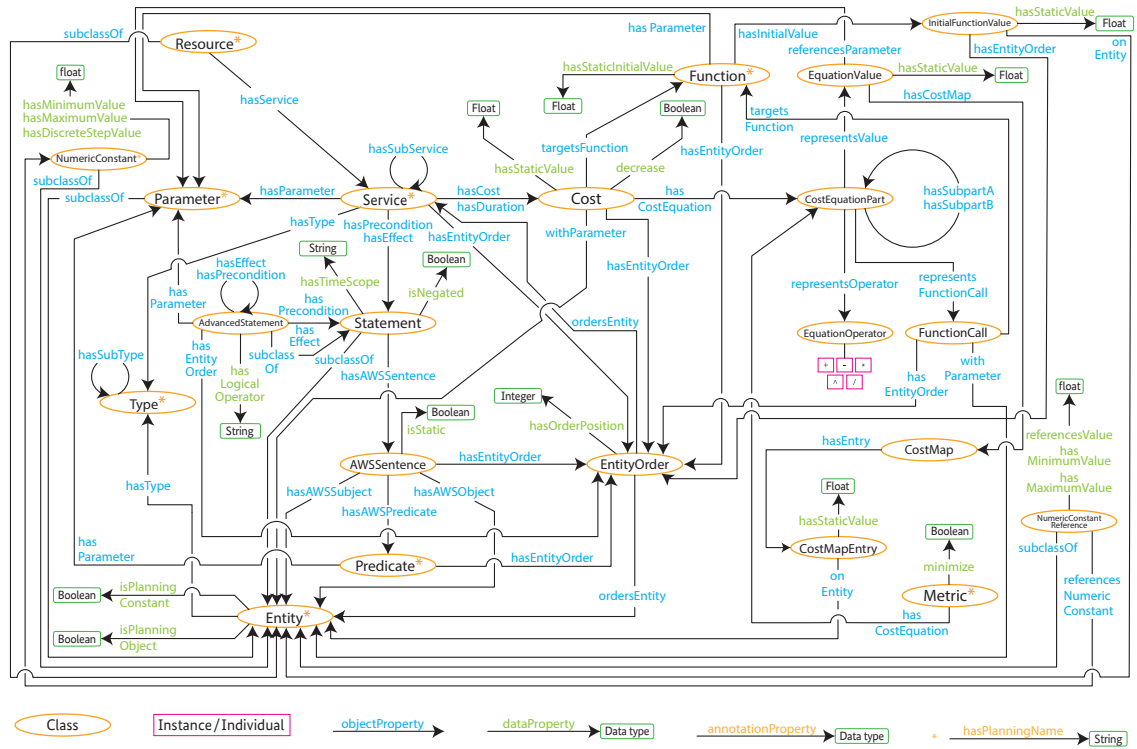


Figure 1: Overview of the Plato model for representing semantic planning information in ontologies. Due to space constraints, inverse object properties and some details are omitted.

from which planner- and user-specific artifacts can be generated. Figure 1 gives an overview. Plato covers actions, called services in the schema, with preconditions, effects, costs, and durations, as well as resources, types, entities, parameters, predicates, metrics, numeric constants, and supplementary transformation information required by R8.

The central action-like concept is *Service*. A service represents a plannable step that can later be transformed into a planning action. The annotation property *hasPlanningName* assigns the name used in generated domains to services and other constructs intended for PDDL translation. Services can be attached to resources such as machines through *isProvidedBy* and *Resource*; this relation can guide domain composition, for example by omitting services of selected resources. Services can also be assigned a *Type* via *hasType*. Types have names and may be organized hierarchically through *hasSubType*, enabling generalization and specialization across typed constructs. A service may reference sub-services through *hasSubService*, capturing hierarchical knowledge without forcing it into flat PDDL actions.

Services can have parameters through

hasParameter and *Parameter*. Parameters use the same planning-name and type properties as services. Multiple parameters can be ordered with *EntityOrder*. Ordering and the common *Entity* abstraction for objects and constants are described in Section 4.2. Applicability and effects are modeled with statements based on the Atomic World State (AWS) concept from Malburg’s approach (Malburg et al., 2023), using *hasPrecondition* and *hasEffect*; Section 4.3 explains this concept. Costs and durations are attached with *hasCost* and *hasDuration*, which reference individuals of *Cost*; the numeric planning constructs are described in Section 4.4.

4.2 Entities, Ordering, and Schema Simplification

The *Entity* class defines planning constants and objects through *hasPlanningName* and *hasType* and distinguished by the data properties *isPlanningConstant* and *isPlanningObject*. As can be seen in Figure 1, *Entity* is also a superclass for planning-context elements such as parameters, resources, and numeric constants. This

common abstraction keeps the ontology compact: instead of defining domains and ranges on every specialized class, shared properties are defined on `Entity` and inherited by subclasses. Each entity can have a type and a name. Entities can also be ordered with `EntityOrder`, which associates an entity with a numeric index for a specific use, such as an action parameter, via `hasOrderPosition`.

4.3 Preconditions, Effects, and World-State Descriptions

Service preconditions and effects are represented by `Statement` individuals attached through `hasPrecondition` or `hasEffect`. A statement represents a fact about the world state, such as one block being stacked on another. It can be negated with `isNegated` and assigned a time scope through `hasTimeScope`, indicating whether it holds at an action's start, end, or over its whole duration. Thus, the same semantic model can support classical and temporal planning.

Each statement references one `AWSSentence` through `hasAWSSentence`. Such a sentence consists of a subject, predicate, and optional objects via `hasAWSSubject`, `hasAWSPredicate`, and `hasAWSObject`. The predicate is represented by `Predicate` and has a name and optionally ordered parameters, separating reusable predicate definitions from concrete realizations. Subject and objects represent planning objects, constants, or service parameters through `Entity`; the subject corresponds to the first predicate argument and the objects to later arguments. This enables intuitive predicate calls, for example `blockA (subject), on (predicate), blockB (object)`.

`AWSSentence` individuals can also be marked as static facts through `isStatic`. Static sentences describe domain facts that do not change during planning and can be modeled once rather than repeated in every problem. During PDDL translation, they are composed into the problem file's initial state.

Advanced Statements Basic statements cover many preconditions and effects, but practical domains often require richer logical structures. Plato therefore provides `AdvancedStatement` as a subclass of `Statement`. Advanced statements combine basic or other advanced statements using conjunction, disjunction, implication, universal statements, and existential statements. Complex applicability conditions can therefore be modeled in the ontology rather than manually encoded in generated PDDL.

This design supports reuse and maintenance. Recurring logical conditions can be modeled once and referenced by several services, and nested expressions can be built compositionally from smaller statements. Changes to shared constraints, such as resource-availability or safety rules, can then be made in the semantic source model and propagated during transformation.

Advanced statements are attached to services with the same `hasPrecondition/Effect` properties, which also attach nested statements. Logical operators are specified with `hasLogicalOperator`. For existential or universal preconditions/effects, `hasParameter` defines the type of objects or constants covered. The Blocks World example ontology in the Plato repository illustrates this feature with a universal precondition.

4.4 Numeric Planning

In many planning settings, finding any valid plan is insufficient: plans must be optimized for criteria, such as cost, energy consumption, resources, or other objectives. Plato represents this information through an expressive cost system. Because the system is extensive, this paper gives only an overview; the full specification is available in the repository.

Numeric functions are represented by `Function`. These individuals are translated directly to PDDL functions and can have ordered parameters and initial values represented by `InitialFunctionValue`. Service costs are mapped to functions so that action effects can change their values. A service can reference one or more `Cost` individuals through `hasCost`; a duration is a special cost attached with `hasDuration` and not mapped to a function. Costs may be static values or cost equations, i.e., arithmetic expressions that may depend on service parameters. Cost equations follow binary expression trees (Issel, 1984), whose nodes represent operators, values, or function calls. Plato can therefore express weighted sums and parameter-dependent costs. If a cost depends on a non-numeric parameter, `CostMap` maps parameter values to numbers, for example when resources or materials imply different costs. The repository includes a 3D-printing example where cost maps assign numeric values to PDDL constants.

Optimization objectives are represented by `Metric`. A metric references a cost equation and specifies minimization or maximization. Multiple metrics can be stored in Plato and one can be selected during transformation.

Numeric Constants and Discretization Continuous numeric values are challenging for domain generation because many planners lack unrestricted numeric reasoning and continuous ranges can lead to infeasible search spaces. Plato therefore provides `NumericConstant`. A numeric constant defines a minimum, maximum, and discretization step via `hasMinimumValue`, `hasMaximumValue`, and `hasDiscreteStepValue`. This lets modelers describe numeric domains semantically while providing the information needed to generate discrete alternatives for planners with limited numeric support.

To avoid manually enumerating generated numeric values, Plato includes `NumericConstantReference`. References point to either a single value or a value range of a numeric constant. Thus, the source ontology stays compact while transformation derives the concrete discrete representation required by the planner.

Overall, Plato separates domain knowledge from planner-specific representations. Predicate definitions, parameter definitions, type hierarchies, static facts, advanced preconditions, cost functions, and metrics become reusable semantic objects that can be transformed automatically. This reduces duplication, localizes changes, and lets one semantic knowledge base serve as the source for different planning-domain representations.

5 TRANSFORMATION AND TOOL SUPPORT WITH KTF

Plato is intended as a semantic source model for planning knowledge. To operationalize it, Plato ontologies must be transformed into artifacts usable by planning tools and related semantic infrastructures. This is provided by the KTF, an open-source framework available on GitHub³ under the MIT license (KTF, 2026). In the context of Plato, KTF translates and composes planning knowledge from Plato ontologies into PDDL domain and problem files, can translate the same knowledge into other semantic structures such as Asset Administration Shell (AAS) (Industrial Digital Twin Association and Plattform Industrie 4.0, 2024), and supports analysis and validation of Plato models and generated domains (KTF, 2026).

The Transformation can be configured to steer composition and translation. Planning objects, initial-state statements, and goal-state statements can be modeled directly in the ontology or supplied through the configuration. Thus, a Plato

ontology can represent a complete problem or act as a reusable domain-level knowledge base from which different problem instances are generated by changing the configuration. The configuration can also address target-planner capabilities. For example, `removeNegativePreconditions` replaces negative preconditions with predicates representing the negated statement, and `removeNumericExpressions` precalculates numeric cost expressions and replaces them with function calls. Planner limitations can therefore be handled during transformation rather than encoded manually in the semantic model.

KTF also validates Plato models while the Plato consumer parses an ontology and maps it to the intermediate planning-domain representation. The consumer can issue warnings and errors for missing required relations, inconsistent references, incomplete ordering information, or constructs that cannot be interpreted unambiguously. This feedback helps modelers detect semantic modeling problems before they appear in generated PDDL artifacts.

Finally, KTF supports statistical analysis of Plato models. It can export domain-level statistics, such as numbers of actions, and action-level statistics, such as numbers of preconditions. These statistics help inspect domain size and structure, compare versions, and identify unusually complex actions. KTF is available via a command-line interface and a Docker container with a REST API, enabling integration into existing environments. Together, Plato and KTF provide both an ontology schema and an executable toolchain for validation, transformation, and analysis.

6 EVALUATION

The evaluation investigates whether Plato can represent the planning information identified in Section 3 and whether this information can be operationalized by automatic transformation. It is a proof of concept using four example ontologies. For each ontology, PDDL domain and problem files were generated with KTF v1.0.0, validated, and solved with off-the-shelf PDDL planners. The ontologies, generated PDDL files, and plan files are available in the Plato GitHub repository.

Specifically the evaluation addresses four questions: whether Plato covers the requirements from Section 3; whether KTF transforms Plato ontologies into syntactically valid and semantically sound PDDL; whether characteristic ontology constructs appear faithfully in the generated PDDL; and whether the generated models are practically usable, as shown

³<https://github.com/user140613/KTF>

by solvable planning tasks.

6.1 Example Ontologies

The evaluation uses four ontologies covering different parts of Plato. Except for the SharedProduction ontology, all ontologies were specifically created by us for the evaluation.

BlocksWorld. The first ontology models the classical Blocks World domain. It is a baseline for typed entities, predicates, ordered parameters, actions, preconditions, and effects, and it also includes advanced preconditions such as universal conditions. The generated PDDL contains the expected actions for picking up and putting down blocks and predicates such as `On`, `OnGround`, `TopClear`, and `Holding`.

TemporalBlocksWorld. The second ontology extends Blocks World with temporal information. It evaluates whether time-scoped statements and durations in Plato are translated into temporal PDDL. The generated domain represents the classical actions as durative actions with explicit durations and temporally scoped preconditions and effects.

SharedProduction. The third ontology models a shared production environment between the research factories SmartFactoryOWL⁴ and SmartFactoryKL⁵. It was designed in the [redacted for blind review] project to study PDDL-based AI planning for production processes spanning multiple factories. The ontology includes production and transport resources, warehouses, 3D printers, cameras, materials, workpieces, quality checks, and cost-related functions. Two generated PDDL variants are considered: one preserves numeric action-cost expressions, while the other configures KTF to substitute them with precalculated costs for use with a non-numeric planner. This evaluates generation for different planner capabilities.

3DPrinting. The fourth ontology models a simplified 3D-printing process with printers, materials, print times, nozzle temperatures, energy costs, material costs, and a metric combining cost terms. It specifically evaluates numeric constants and discretization. Numeric ranges, such as nozzle temperatures and estimated print times, are represented in the ontology and transformed into discrete PDDL constants and corresponding numeric function values that influence costs.

6.2 Requirement Coverage

The four ontologies jointly cover all requirements from Section 3. Typed entities and type hierarchies

⁴<https://www.smartfactory-owl.de/en/>

⁵<https://smartfactory.de/>

(R1), predicates with ordered parameters (R2), and actions with parameters, preconditions, and effects (R3) are covered by all examples. Temporal action information in R3 is covered by TemporalBlocksWorld. Arithmetic expressions for costs and durations (R4), optimization metrics (R5), and functions with initial numeric values (R6) are covered by SharedProduction and 3DPrinting. Numeric discretization (R7) is covered by 3DPrinting. Planner-specific PDDL generation (R8) is covered by all examples and especially by the two SharedProduction variants. All constructs in the example ontologies were successfully transformed into PDDL by KTF.

The examples, therefore, exercise both symbolic and numeric parts of Plato. BlocksWorld and TemporalBlocksWorld cover basic and temporal action modeling. SharedProduction covers a larger production domain with factories, machines, transport resources, functions, costs, and metrics. The 3DPrinting ontology adds numeric constant discretization.

6.3 Transformation and Validation Results

For all four ontologies, KTF generated valid PDDL domain and problem files without manual post-processing, and the generated domains contain the expected constructs. The artifacts were validated in two steps. First, all generated domain and problem files were validated successfully for syntactical correctness with VAL version 4⁶; VAL reported no problems. The authors also manually checked that Plato constructs were reflected as intended in the generated PDDL. Second, all planning tasks were solved with planners suitable for the respective PDDL fragments. BlocksWorld and SharedProduction with substituted numeric expressions were solved with Fast Downward v22.12⁷. TemporalBlocksWorld was solved with Temporal Fast Downward v0.4⁸. SharedProduction with numeric expressions and 3DPrinting were solved with ENHSP-20 v0.21.0⁹. In every case, a plan was generated that was successfully validated by Val regarding the associated domain and problem.

These results indicate that the generated PDDL artifacts are syntactically valid and usable by planning tools. They also show that Plato can act as a source model for different PDDL fragments: classical, temporal, numeric, and planner-simplified variants can be generated depending on the modeled information and the selected KTF configuration.

⁶<https://github.com/KCL-Planning/VAL>

⁷<https://github.com/aibasell/downward>

⁸<https://github.com/roveri-marco/tfd>

⁹<https://github.com/hstairs/enhsp>

7 DISCUSSION

The evaluation shows that Plato represents the selected requirements for semantic planning knowledge and that KTF can transform this knowledge into executable PDDL. The main contribution, however, is not merely generating PDDL from an ontology. It is the separation of planning-knowledge representation and maintenance from planner-oriented artifacts. This section discusses the implications, distinctive features, and limitations of this separation.

7.1 Benefits Compared to Direct PDDL Modeling

Direct PDDL modeling requires domain experts to encode planning knowledge in a syntax designed primarily for planners, which complicates maintenance when an application domain changes. Plato shifts the primary representation into an ontology. Planning concepts such as types, entities, and actions become explicit semantic objects that can be referenced, reused, and updated independently of any PDDL file.

This separation improves maintainability in three ways. First, recurring concepts such as parameter and predicates definitions, and logical expressions can be modeled once and reused. Second, changes can be made at the semantic level rather than in textual PDDL; updates to a parameter type, predicate, or cost expression can be propagated through transformation to individual PDDL artifacts. Third, the ontology provides an explicit structure for validation and analysis before planner-specific artifacts are generated.

Plato also decouples semantic modeling from planner-specific restrictions. Planners differ in support for numeric expressions, negative preconditions, temporal constructs, and advanced logical conditions. When knowledge is maintained directly in PDDL, such restrictions often shape the domain model itself. With Plato, they can be shifted to the transformation layer. KTF can generate different PDDL variants from the same semantic source, for example by preserving numeric expressions or replacing them with precomputed function calls. The ontology remains the stable knowledge representation, while generated artifacts are adapted to the selected planner.

7.2 Distinctive Features of Plato

A first distinctive feature is support for expressive preconditions. Many domains require more than conjunctions of atomic predicates. Plato supports advanced statements, including conjunctions, disjunctions, implications, universal preconditions, and exist-

tential preconditions. Since these expressions are ontology individuals, they can be composed and reused across services, for example where actions share safety, resource-availability, or invariant constraints.

A second feature is the treatment of costs, functions, and metrics as first-class modeling elements. Plato does not restrict actions to static costs; it allows parameter-dependent functions, cost maps, arithmetic expressions, and optimization metrics. This is important because many applications seek plans optimized for time, energy, monetary cost, resource efficiency, or similar criteria. Explicit ontology-level representation makes these optimization aspects easier to inspect and maintain.

A third feature is numeric discretization. Continuous numeric domains are difficult for many planners because they often support only restricted numeric fragments or finite symbolic object sets. Plato allows numeric constants to be modeled through ranges and step sizes, and KTF can generate discrete planning entities and numeric values from this compact representation. Modelers can therefore describe numeric knowledge at a higher level while still producing planner-compatible representations.

Finally, Plato is a generic planning ontology rather than a domain-specific model. This distinguishes it from approaches targeting a particular factory, robot, or service-composition setting. The evaluated ontologies indicate that the same schema can cover classical benchmarks, temporal variants, production scenarios, and numeric domains. The SharedProduction example further suggests applicability beyond artificial examples, including distributed production workflows.

7.3 Limitations and Open Issues

The evaluation has several limitations. The example ontologies are small and curated to cover requirements and demonstrate selected modeling features. They show feasibility but not sufficiency for all relevant planning domains. Larger, independently developed ontologies are needed for broader assessment.

A second limitation is that transformation success does not prove usability at scale. The generated PDDL files are valid and solvable, but this does not show how easily domain experts can create, understand, and maintain large Plato ontologies. Ontologies with many actions, parameters, expressions, and references may become difficult to navigate. Planning time, plan quality, and scalability were also not systematically investigated. Future work should therefore examine authoring support, visualization, validation feedback, modeling guidelines, planning performance, and plan quality.

A third limitation concerns the proof-of-concept focus. The current evaluation emphasizes requirement coverage, transformation feasibility, and planner usability. It does not measure human authoring effort, error rates, maintainability, or model evolution compared with directly written PDDL. Such empirical evidence is needed to quantify Plato’s practical benefits as a knowledge-engineering tool.

Open technical questions also remain. Plato currently covers commonly used planning constructs relevant for common PDDL generation rather than the full PDDL family. This keeps the ontology manageable but may require future extensions for additional paradigms or planner-specific constructs. Moreover, although KTF can generate different PDDL variants, further work should systematically characterize which Plato constructs can be transformed into which planner-supported fragments without semantic loss.

Despite these limitations, the evaluation supports the main claims. Plato represents selected constructs for semantic planning knowledge; KTF transforms them into different PDDL variants; generated models can be processed by suitable planners; and resulting plans are valid. The examples cover both classical benchmarks and practically motivated production scenarios. Plato should therefore be understood as a semantic source model that makes planning knowledge more maintainable, reusable, and adaptable before transformation into planner-specific artifacts. The remaining challenge is to demonstrate these advantages in larger domains, with more users, and across more diverse planning scenarios.

8 CONCLUSION AND FUTURE WORK

This paper presented Plato, a generic ontology schema for representing semantic planning knowledge independent of a specific planning domain. Plato addresses the need for a maintainable source representation by modeling typed entities, predicates, parameters, actions, preconditions, effects, functions, costs, metrics, and numeric constants as semantic objects. It thereby separates the maintenance of planning knowledge from the planner-specific artifacts generated from it.

Plato’s development was guided by a compact requirement baseline derived from common PDDL constructs, prior semantic-to-PDDL transformation work (Malburg et al., 2023), and practical planner-support considerations. The proof-of-concept evaluation showed that Plato covers this baseline across classical and temporal Blocks World variants, a

shared production scenario, and a 3D-printing example with numeric discretization. Using KTF, these ontologies were transformed into PDDL domain and problem files. The generated tasks were solved by suitable planners and validated with VAL, providing initial evidence that Plato is a feasible semantic source model for automatic planning-domain generation.

Future work should extend the evaluation beyond curated examples. Larger case studies are needed to assess scaling with domain complexity and support for evolving planning knowledge. Further planner-based validation should investigate more complex domains, planner configurations, and semantic equivalence of alternative generated PDDL fragments. Future work should also examine authoring effort, error rates, and maintainability compared with directly written PDDL. Finally, Plato should be compared systematically with alternative semantic representations to clarify which representation best fits which modeling and deployment context.

ABBREVIATIONS

The following abbreviations are used in this manuscript:

AAS	Asset Administration Shell
AI	Artificial Intelligence
AWS	Atomic World State
IOPEs	Inputs, Outputs, Preconditions, and Effects
KEWI	Knowledge Engineering Web Interface
KTF	Knowledge Transformation Framework
LLM	Large Language Model
OCL	Object Constraint Language
OWL	Web Ontology Language
PDDL	Planning Domain Definition Language
RDFS	RDF Schema
RDF	Resource Description Framework
UML	Unified Modeling Language
XSDL	XML Schema Definition Language

ACKNOWLEDGEMENTS

[redacted for blind review]

USE OF GENERATIVE AI

ChatGPT (OpenAI, 2026) was used to polish the wording and correct spelling and grammar mistakes for the sections in this paper. Each section was first written by the authors without the use of any AI tools before being passed to the Large Language Model (LLM) for polishing. All LLM-adjusted texts were carefully checked sentence by sentence for accuracy by the authors. No graphs, figures, code, data, or citations were AI-generated.

REFERENCES

- Alnazer, E. and Georgievski, I. (2023). Understanding real-world AI planning domains: A conceptual framework. In Aiello, M., Barzen, J., Dustdar, S., and Leymann, F., editors, *Service-Oriented Computing - 17th Symposium and Summer School, SummerSOC 2023, Heraklion, Crete, Greece, June 25 - July 1, 2023, Revised Selected Papers*, volume 1847 of *Communications in Computer and Information Science*, pages 3–23. Springer.
- Alterovitz, R., Koenig, S., and Likhachev, M. (2016). Robot planning in the real world: Research challenges and opportunities. *AI Mag.*, 37(2):76–84.
- Ammar, S. and Bhiri, M. T. (2018). Automatic planning: From event-b to PDDL. In Abdelwahed, E. H., Belatreche, L., Benslimane, D., Golfarelli, M., Jean, S., Méry, D., Nakamatsu, K., and Ordonez, C., editors, *New Trends in Model and Data Engineering - MEDI 2018 International Workshops, DETECT, MEDI4SG, IWCFS, REMEDY, Marrakesh, Morocco, October 24-26, 2018, Proceedings*, volume 929 of *Communications in Computer and Information Science*, pages 247–254, Cham Switzerland. Springer.
- Bartheye, O. and Jacopin, E. (2009). A real-time pddl-based planning component for video games. In Darken, C. and Youngblood, G. M., editors, *Proceedings of the Fifth Artificial Intelligence and Interactive Digital Entertainment Conference, AIIDE 2009, October 14-16, 2009, Stanford, California, USA*. The AAAI Press.
- Bresina, J. L., Jónsson, A. K., Morris, P. H., and Rajan, K. (2005). Activity planning for the mars exploration rovers. In Biundo, S., Myers, K. L., and Rajan, K., editors, *Proceedings of the Fifteenth International Conference on Automated Planning and Scheduling (ICAPS 2005), June 5-10 2005, Monterey, California, USA*, pages 40–49. AAAI.
- Feigenbaum, E. A. (1977). The art of artificial intelligence: Themes and case studies of knowledge engineering. In Reddy, R., editor, *Proceedings of the 5th International Joint Conference on Artificial Intelligence*. Cambridge, MA, USA, August 22-25, 1977, pages 1014–1029. William Kaufmann.
- Fox, M. and Long, D. (2003). Pddl2.1: An extension to pddl for expressing temporal planning domains. *J. Artif. Intell. Res. (JAIR)*, 20:61–124.
- Green, C. (1969). Application of Theorem Proving to Problem Solving. In *1st IJCAI Proc.*, IJCAI’69, page 219–239, San Francisco, CA, USA. Morgan Kaufmann Publishers Inc.
- Haslum, P. (2006). *Admissible Heuristics for Automated Planning*. PhD thesis, Linköping University, KPLAB - Knowledge Processing Lab, The Institute of Technology.
- Haslum, P., Lipovetzky, N., Magazzeni, D., and Muise, C. (2019). *An Introduction to the Planning Domain Definition Language*. Synthesis Lectures on Artificial Intelligence and Machine Learning. Morgan & Claypool Publishers, California, USA.
- Hatzi, O., Vrakas, D., Nikolaidou, M., Bassiliades, N., Anagnostopoulos, D., and Vlahavas, I. (2012). An integrated approach to automated semantic web service composition through planning. *IEEE Transactions on Services Computing*, 5(3):319–332.
- Helmert, M. (2009). Concise finite-domain representations for PDDL planning tasks. *Artif. Intell.*, 173(5-6):503–535.
- Industrial Digital Twin Association and Plattform Industrie 4.0 (2024). *Asset Administration Shell: Reading Guide*. Industrial Digital Twin Association (IDTA). Accessed 16.05.2026.
- Issel, W. (1984). Aho, a.v., j. e. hopcroft, j. d. ullman: Data structures and algorithms. addison-wesley amsterdam 1983. 436 s. *Biometrical Journal*, 26(4):390–390.
- John, T. and Koopmann, P. (2023). Planning with ontology-enhanced states using problem-dependent rewritings. In Umbrico, A. and Sanfilippo, E. M., editors, *Proceedings of the 1st International Planning and Ontology Workshop co-located with The 33rd International Conference on Automated Planning and Scheduling (ICAPS 2023), Prague, Czech Republic, July 10, 2023*, CEUR Workshop Proceedings. CEUR-WS.org.
- Klusch, M., Gerber, A., and Schmidt, M. (2005). Semantic web service composition planning with owls-xplan. *AAAI Fall Symposium - Technical Report*.
- Kootbally, Z., Schlenoff, C., Lawler, C., Kramer, T., and Gupta, S. (2015a). Towards robust assembly with knowledge representation for the planning domain definition language (pddl). *Robotics and Computer-Integrated Manufacturing*, 33:42–55. Special Issue on Knowledge Driven Robotics and Manufacturing.
- Kootbally, Z., Schlenoff, C., Lawler, C., Kramer, T., and Gupta, S. (2015b). Towards robust assembly with knowledge representation for the planning domain definition language (pddl). *Robotics and Computer-Integrated Manufacturing*, 33:42–55. Special Issue on Knowledge Driven Robotics and Manufacturing.
- KTF (2026). KTF: A framework for semantic knowledge transformation. Submitted for review at KEOD 2026. Authors removed for double-blind review since we are referencing our own work. Anonymized preprint available at <https://github.com/user140613/Preprints>.
- Malburg, L., Klein, P., and Bergmann, R. (2023). Converting semantic web services into formal planning domain descriptions to enable manufacturing process planning and scheduling in industry 4.0. *Eng. Appl. Artif. Intell.*, 126:106727.

- Marrella, A. (2019). Automated planning for business process management. *J. Data Semant.*, 8(2):79–98.
- Martin, D. L., Paolucci, M., McIlraith, S. A., Burstein, M. H., McDermott, D. V., McGuinness, D. L., Parsia, B., Payne, T. R., Sabou, M., Solanki, M., Srinivasan, N., and Sycara, K. P. (2004). Bringing semantics to web services: The OWL-S approach. In Cardoso, J. and Sheth, A. P., editors, *Semantic Web Services and Web Process Composition, First International Workshop, SWSWPC 2004, San Diego, CA, USA, July 6, 2004, Revised Selected Papers*, Lecture Notes in Computer Science, pages 26–42. Springer.
- McCluskey, T. L., Vallati, M., and Franco, S. (2017). Automated planning for urban traffic management. In Sierra, C., editor, *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence, IJCAI 2017, Melbourne, Australia, August 19-25, 2017*, pages 5238–5240. ijcai.org.
- Muscettola, N., Nayak, P. P., Pell, B., and Williams, B. C. (1998). Remote agent: To boldly go where no AI system has gone before. *Artif. Intell.*, 103(1-2):5–47.
- Nabizada, H., Jeleniewski, T., Gehlhoff, F., and Fay, A. (2024). Model-based workflow for the automated generation of pddl descriptions. In *2024 IEEE 29th International Conference on Emerging Technologies and Factory Automation (ETFA)*, pages 01–04.
- OpenAI (2026). Chatgpt (version 5.5). Large language model. Accessed throughout May and June 2026.
- Puttonen, J., Lobov, A., and Martinez Lastra, J. L. (2013). Semantics-based composition of factory automation processes encapsulated by web services. *Industrial Informatics, IEEE Transactions on*, 9:2349–2359.
- Russell, S. and Norvig, P. (2020). *Artificial Intelligence: A Modern Approach (4th Edition)*. Pearson, London, United Kingdom.
- Studer, R., Benjamins, V. R., and Fensel, D. (1998). Knowledge engineering: Principles and methods. *Data Knowl. Eng.*, 25(1-2):161–197.
- Vaquero, T. S., Romero, V., Tonidandel, F., and Silva, J. R. (2007). itsimple 2.0: An integrated tool for designing planning domains. In *International Conference on Automated Planning and Scheduling*.
- Vaquero, T. S., Silva, J. R., Tonidandel, F., and Beck, J. C. (2013). itsimple: towards an integrated design system for real planning applications. *The Knowledge Engineering Review*, 28:215 – 230.
- Wally, B., Vyskočil, J., Novák, P., Huemer, C., Šindelář, R., Kadera, P., Mazak, A., and Wimmer, M. (2019). Production planning with iec 62264 and pddl. In *2019 IEEE 17th International Conference on Industrial Informatics (INDIN)*, volume 1, pages 492–499, New York, NY, USA. IEEE.
- Wickler, G., Chrapa, L., and McCluskey, T. L. (2014). Ontological support for modelling planning knowledge. In Fred, A. L. N., Dietz, J. L. G., Aveiro, D., Liu, K., and Filipe, J., editors, *Knowledge Discovery, Knowledge Engineering and Knowledge Management - 6th International Joint Conference, IC3K 2014, Rome, Italy, October 21-24, 2014, Revised Selected Papers*, Communications in Computer and Information Science, pages 293–312. Springer.